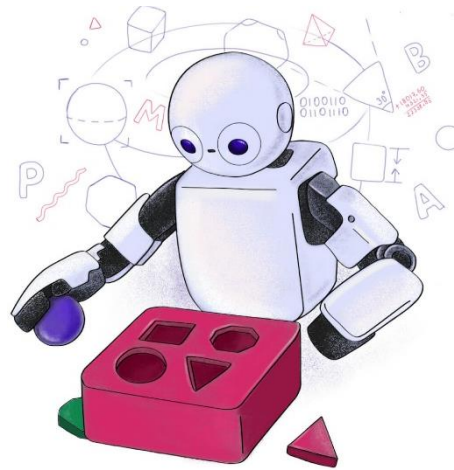


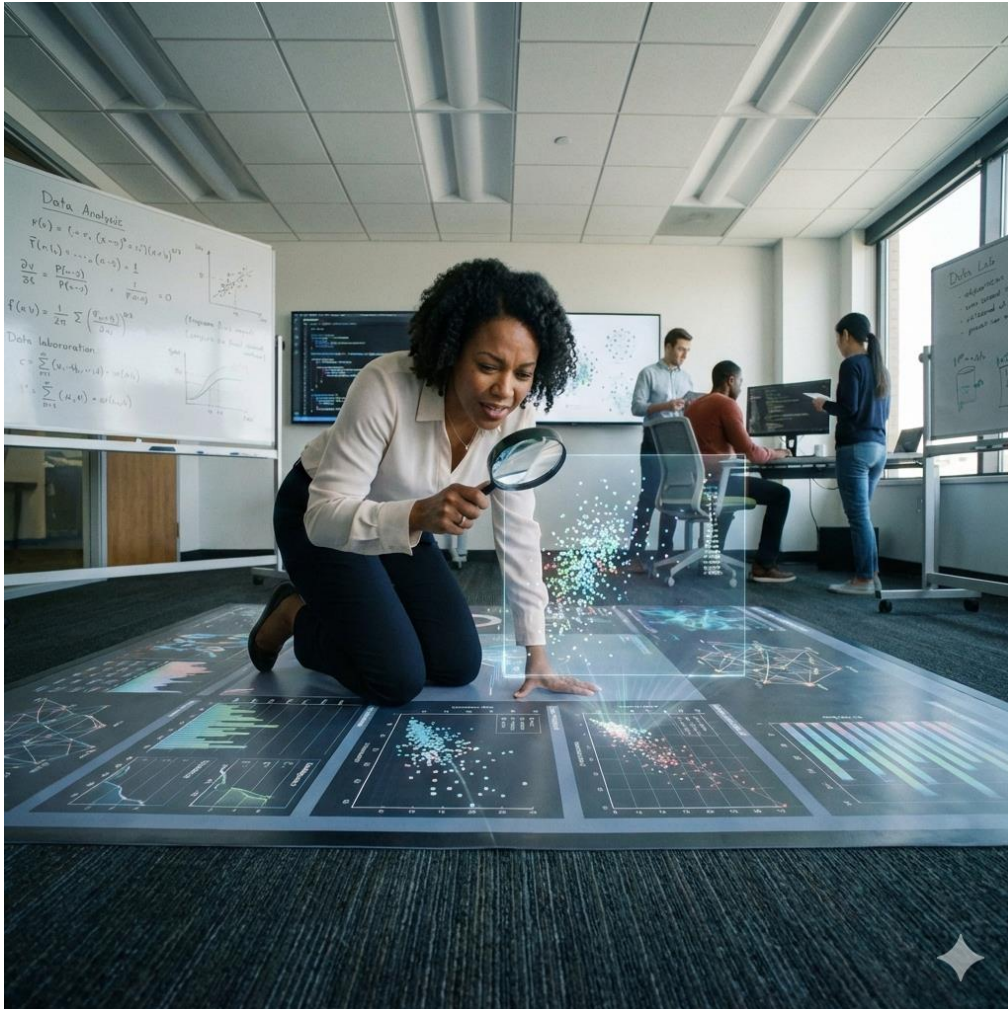
C24 - Inteligência Artificial: Análise Exploratória de Dados (EDA)





Etapa executada
antes da
construção e
treinamento dos
modelos de ML.

Introdução



- É o **primeiro passo** em qualquer projeto de ciência de dados, incluindo ML.
- EDA nos ajudar a identificar:
 - **erros** óbvios nos dados,
 - compreender os **padrões** nos dados,
 - detectar valores **discrepantes, faltantes, e duplicados**,
 - encontrar **relações entre as variáveis** e
 - descobrir quais **variáveis realmente importam**.

Por que realizar análise exploratória?



- A análise exploratória é uma espécie de **curadoria** dos dados.
- Ela garante que o modelo de ML não aprenda com lixo.
- Se o **dado de entrada é ruim**, a **previsão será pior** ainda.

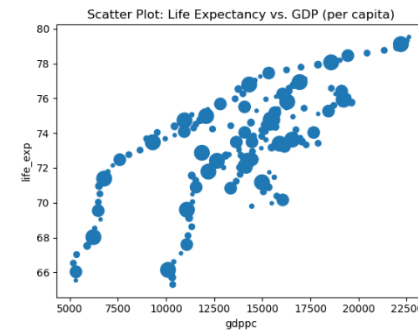
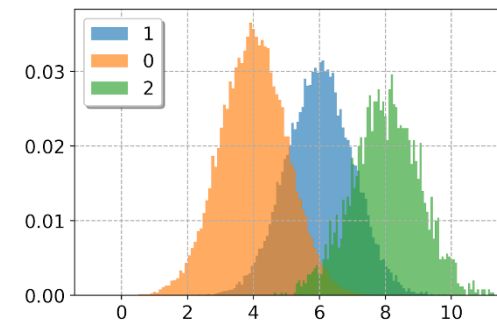
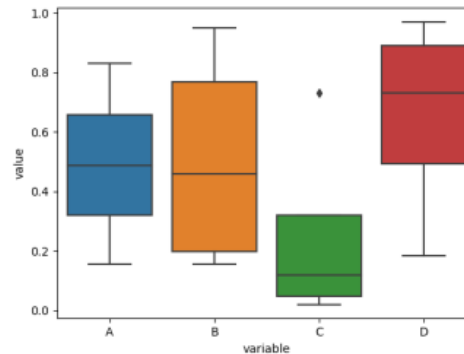
Inspeção visual e estatística

- Utilizamos **ferramentas estatísticas e visuais** como

- Histogramas
- *Heatmaps* de correlação
- *Boxplots*
- Gráficos de barras
- Diagramas de dispersão

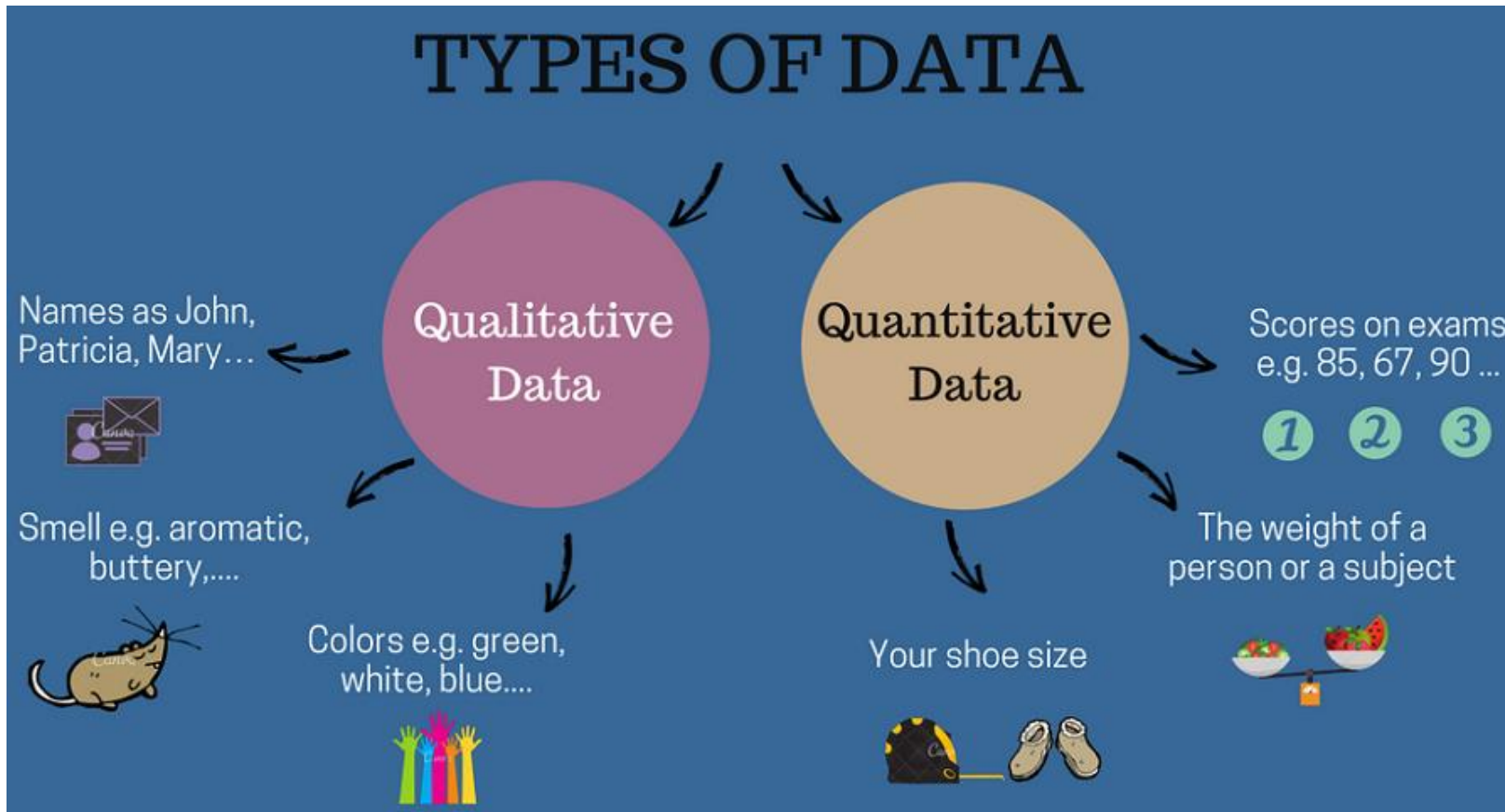
- Bibliotecas mais usadas:

- Pandas
- Numpy
- Scikit-learn
- Matplotlib
- Seaborn
- Pandas *profiling* (Anexo IV)



*Antes de explorarmos os dados, nós
precisamos entender a sua
taxonomia*

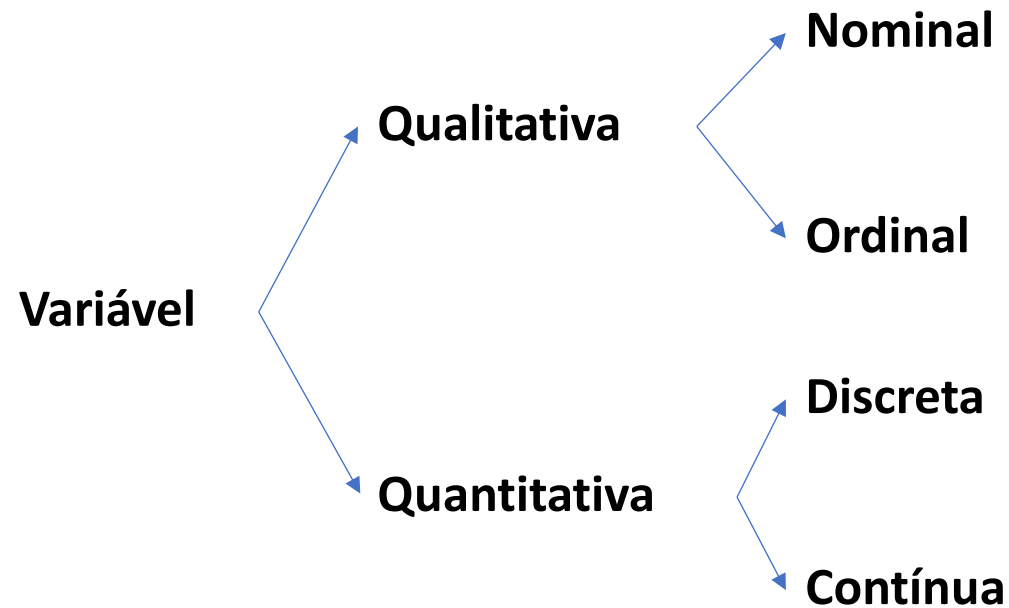
Tipo dos dados



- As variáveis (i.e., atributos ou rótulos) podem ser classificadas em
 - Qualitativas
 - Quantitativas

Tipo dos dados

- As variáveis qualitativas e quantitativas podem ser classificadas da seguinte forma:



Variáveis qualitativas

Também chamadas de categóricas.

Elas descrevem uma **qualidade, característica ou atributo** de algo ou alguém.

- **Nominal:** Não possuem uma ordem ou hierarquia.
 - Exemplos: Cor dos olhos, estado civil, gênero, nacionalidade, profissão.
- **Ordinal:** Existe uma hierarquia ou ordem lógica.
 - Exemplos: Febre (baixa, média, alta), tamanho de uma camiseta (P, M, G), escolaridade (Ensino Fundamental, Médio, Superior), meses do ano.
- Embora as ordinais possam ser ordenadas, operações aritméticas não são aplicáveis.
 - Não faz sentido somar, subtrair tamanhos de camisetas.

Variáveis quantitativas

Também chamadas de **variáveis numéricas**.

São números reais.

Podem ser **ordenadas e usadas em operações** aritméticas (e.g., soma, subtração, etc.).

Possuem **unidade de medida**.

- **Discreta:** São **contagens**, resultando em números inteiros.
 - Exemplos: Quantidade de carros em um estacionamento, número de filhos, número de acessos em um site.
- **Contínua:** São **medições** que podem assumir qualquer valor dentro de um intervalo.
 - Exemplos: Altura, peso, temperatura, tempo de resposta de uma API.

Etapas de exploração dos dados

Na sequência, veremos as seguintes etapas de exploração dos dados:

- Inspeção inicial dos dados
- Limpeza dos dados
- Relação entre variáveis

Inspeção inicial dos dados

Inicialmente, verificamos algumas informações básicas dos dados.

Em geral, após carregarmos os dados, usamos **atributos e métodos** da biblioteca **pandas**.

- Dimensão dos dados: linhas x colunas
 - `df.shape`
- Tipos de dados: numéricos, categóricos, etc.
 - `df.dtypes`
- Estatísticas básicas: média, min, max, quartis, mediana, desvio padrão, etc.
 - `df.describe()`

Limpeza dos dados

Alguns problemas comuns que podemos encontrar nos dados são:

- Valores irrelevantes
- Duplicatas
- Valores faltantes
- Tipos inconsistentes (e.g., número como texto, datas quebradas)
- *Outliers*

Vamos ver como lidar com cada um deles na sequência.

Removendo valores irrelevantes

- A ideia é manter colunas que ajudem o objetivo do problema e **remover as colunas que atrapalham**.
 - `df.drop(['Column 1', 'Column 2', 'Column N'], axis=1)`
- Alguns motivos para eliminar colunas:
 - IDs,
 - *timestamps*,
 - coluna com valores constantes,
 - coluna com muitos valores faltantes,
 - colunas redundantes, etc.

Removendo duplicatas

- Quantas linhas duplicadas existem?
 - `df.duplicated().sum()`
- Encontrar e remover linhas duplicadas:
 - `df.drop_duplicates(inplace=True)`
 - OBS.: ``inplace=True`` faz a alteração diretamente no próprio df, sem criar um novo DataFrame.
- **OBS.:** Para encontrar e remover colunas duplicadas, basta transpor o DataFrame.

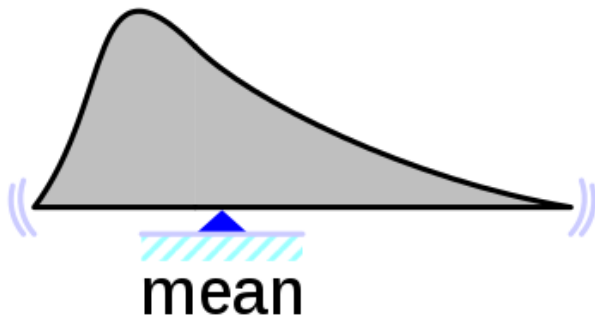
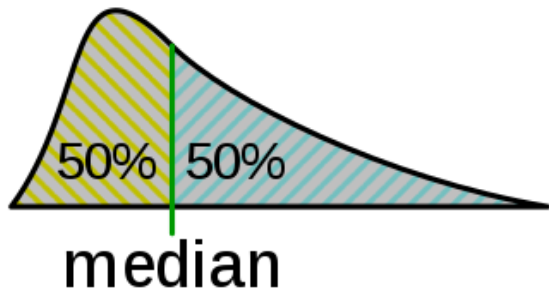
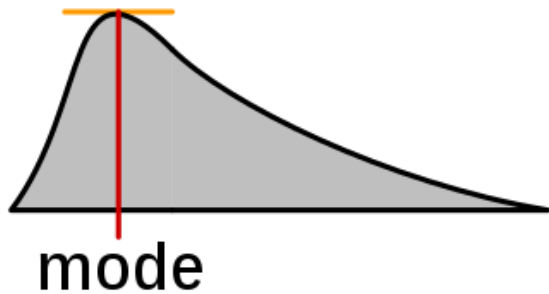
Tratando valores faltantes

- Quantos valores estão faltando?
 - `df.isnull().sum()`
- Técnicas para o tratamento de valores ausentes:
 - Remover linhas ou colunas
 - Preencher (imputar) com a média, a moda ou a mediana
 - Preenchimento inteligente: Técnicas mais avançadas. Elas olham para a **relação entre as colunas** para "adivinhar" o valor que falta.
 - Anexo I

Técnicas para o tratamento de valores ausentes

- **Remover linhas ou colunas:**
 - `df.dropna(inplace=True)`
 - OBS.: altera o `df` original.
- Essa técnica não é boa se o número de linhas a serem removidas é muito grande em relação ao tamanho do *dataset*.
- Vamos usar técnicas que **imputam valores** aos faltantes.

Técnicas para o tratamento de valores ausentes



- **Preencher com a média, a moda ou a mediana**
 - **Média:** É a soma de todos os valores dividida pela quantidade total de elementos.
 - Sensível à *outliers*.
 - Usada com variáveis **numéricas**.
 - **Mediana:** É o valor que ocupa a posição central de um conjunto de dados ordenado. Divide os dados em 50% acima e 50% abaixo.
 - Insensível a *outliers*.
 - Usada com variáveis **numéricas**.
 - **Moda:** É o valor que aparece com a maior frequência no *dataset*.
 - Insensível a *outliers*.
 - Usada com variáveis **numéricas e categóricas**.

Técnicas para o tratamento de valores ausentes

- **Preencher com a média ou mediana:**

- `num_cols = df.select_dtypes(include="number").columns`
 - Seleciona **apenas as colunas numéricas** do DataFrame (tipos como int64, float64, etc.).
 - Retorna uma **lista com os nomes** das colunas numéricas.

Média

- `df[num_cols] = df[num_cols].fillna(df[num_cols].mean())`
 - Pega todas as colunas numéricas e preenche os valores faltantes de cada uma com a sua média.

Mediana

- `df[num_cols] = df[num_cols].fillna(df[num_cols].median())`
 - Pega todas as colunas numéricas e preenche os valores faltantes de cada uma com a sua mediana.

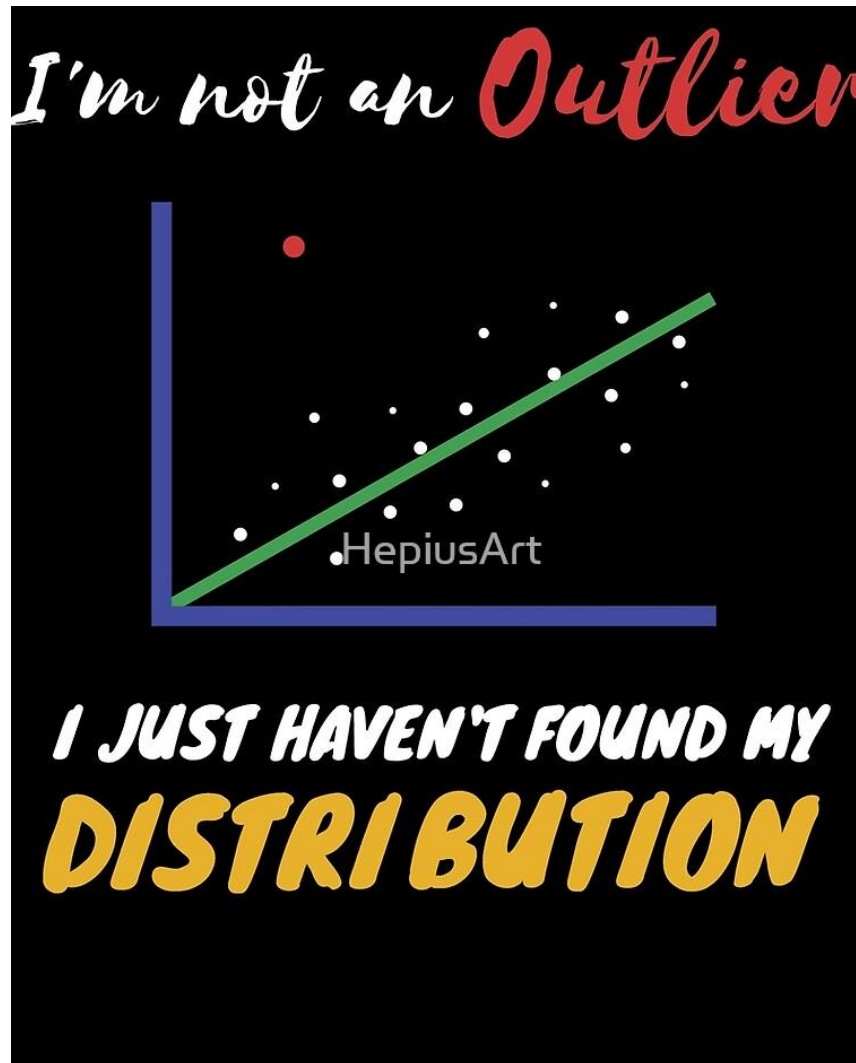
- Os métodos `mean()` e `median()` **ignoram** valores faltantes.

Técnicas para o tratamento de valores ausentes

- **Preencher com a moda:**

- `num_cols = df.select_dtypes(include="number").columns`
 - Obtém os nomes apenas das colunas numéricas do DataFrame.
- `df[num_cols] = df[num_cols].fillna(df[num_cols].mode().iloc[0])`
 - Seleciona as colunas numéricas e preenche os valores faltantes de cada uma com a sua moda.
- **OBS.1:** A **moda pode ter mais de um valor**. Assim, em geral, usamos o primeiro valor (i.e., `iloc[0]`).
- **OBS.2:** A moda pode ser usada para preencher valores faltantes de variáveis categóricas. Portanto, para selecionar colunas categóricas, troque `include="number"` por `include="object"`.

O que é um *outlier*?

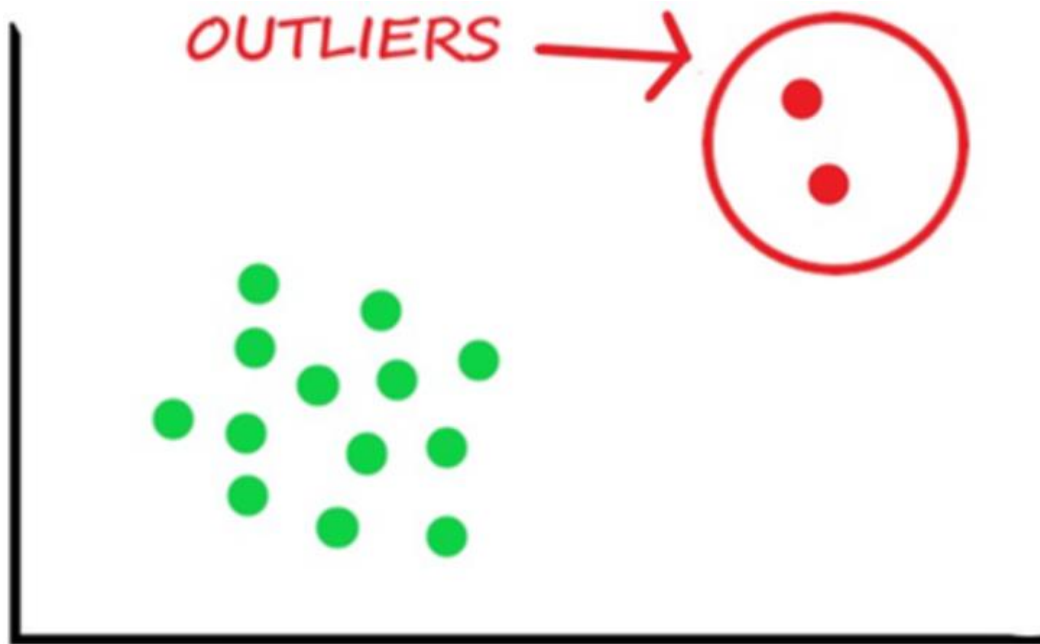


Um *outlier* é uma observação que **foge muito do padrão geral dos dados**, ou seja, um valor extremamente diferente da maioria.

O que é um *outlier*?



O que é um *outlier*?



- Um *outlier* pode ser
 - Um erro de medição (e.g., sensor defeituoso),
 - Erro de registro (e.g., digitação),
 - Erro de processamento dos dados (e.g., conversão errada de unidade),
 - um valor **raro**, **mas válido** (e.g., uma fraude),
 - etc.

O que eles causam?

- Distorcem estatísticas
 - Afetam média, desvio padrão e correlações.
- Prejudicam visualizações
 - Esticam a escala e escondem padrões reais.
- Impactam modelos de ML, especialmente os baseados em distância
 - Geram modelos enviesados e de pior desempenho.
- Porém, podem revelar informação importante
 - *Outliers* podem **revelar evento raro relevante** como fraude, falha em sistemas, ataques cibernéticos.

O que eles causam?



r/Showerthoughts

Posted by u/ [redacted] • 2h

Bill Gates and Mark Zuckerberg are so rich the average income of Harvard dropouts is probably significantly higher than Harvard graduates.



A média salarial de quem largou Harvard é maior que a de quem se formou.

- Porém, largar a faculdade, provavelmente, não deixará vocês bilionários.
- O dado está correto, mas ele é uma **exceção** (não um erro) tão grande que **destrói a representatividade da média**.

O que eles causam?



Se estivéssemos medindo a temperatura de um servidor e um sensor marcasse 500°C por um segundo, isso é um erro ou uma exceção que deve ser investigada?

- *Outliers* são exceções.
- Porém, exceções não são **necessariamente erros**.
- Nunca devemos removê-los ou ignorá-los sem **investigar**.

Como detectar *outliers*?

- De forma visual:
 - Histograma
 - *Boxplot* com intervalos interquartis (IQR)
- De forma estatística:
 - Intervalo Interquartil (IQR)
 - Z-score (Anexo II)

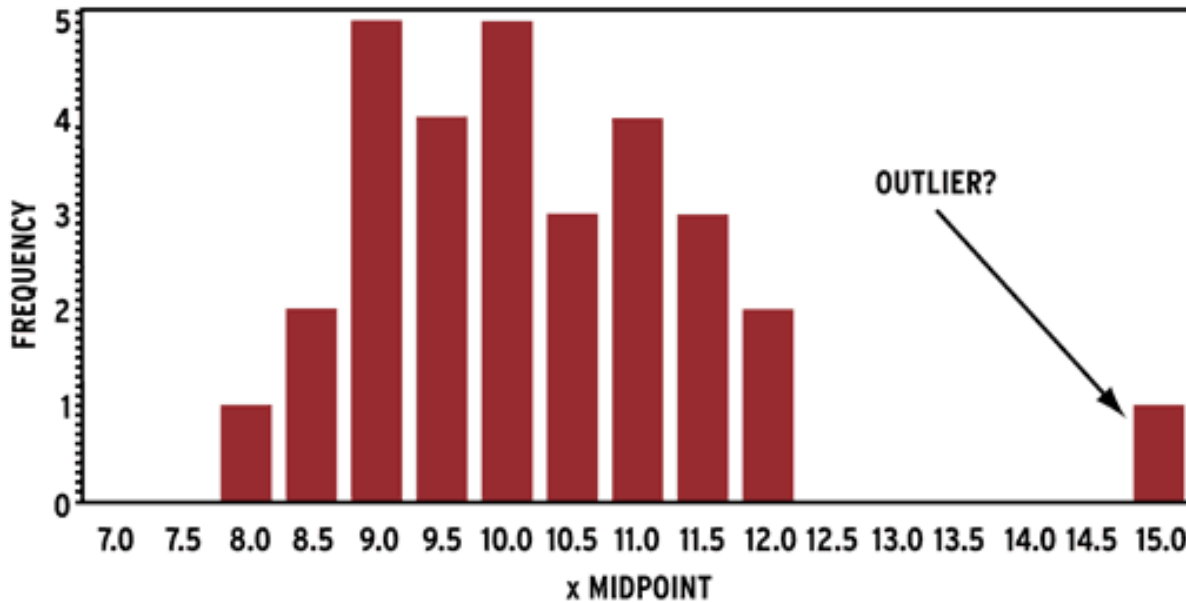
Como detectar *outliers*?

Histograma

- Usando a função `hist` da matplotlib

```
plt.hist(df['Price'], bins=30)
plt.xlabel('Preço')
plt.ylabel('Frequência')
plt.title('Histograma do Preço')
plt.show()
```
- **OBS.:** O parâmetro `bins` ajusta o tamanho das caixas de contagem e é opcional, por padrão, é igual a 10. Funciona bem com valores de 20 a 30.

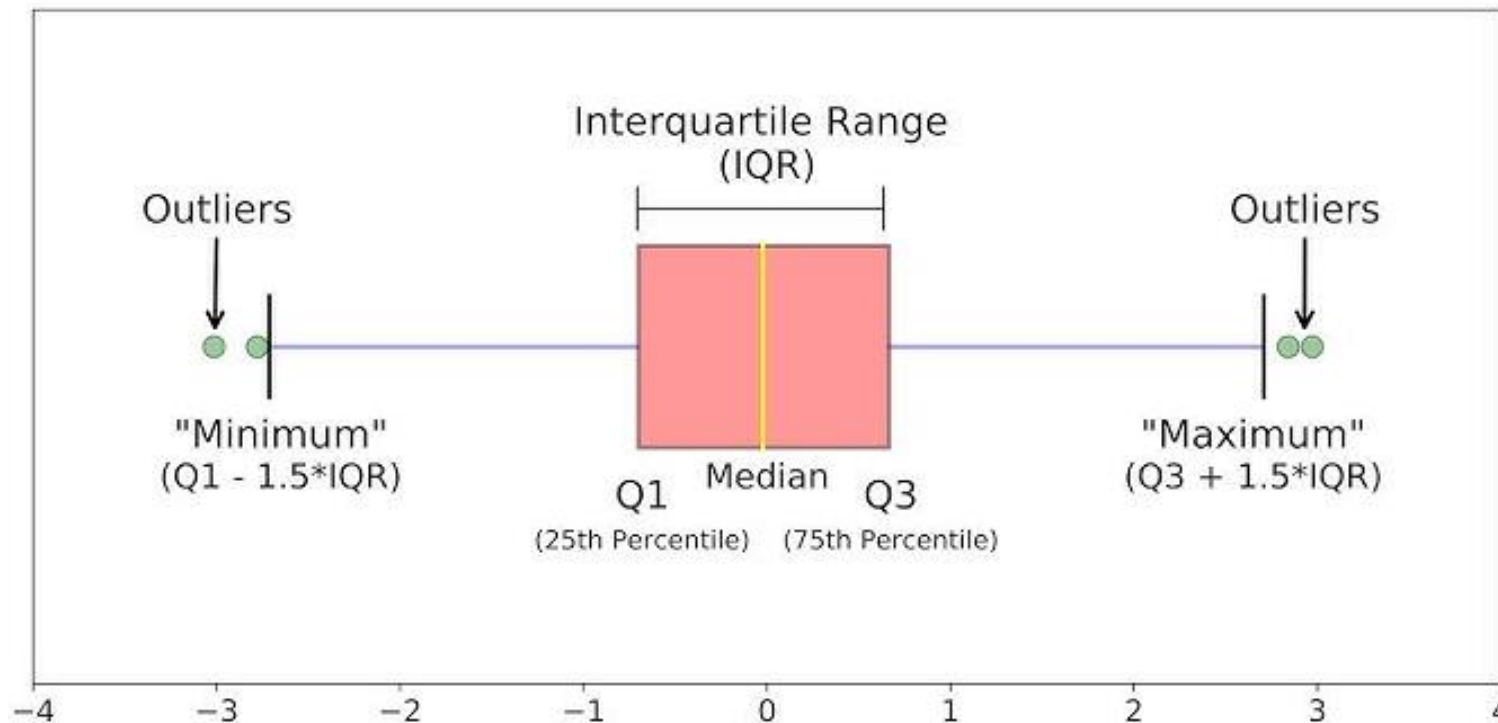
Como detectar *outliers*?



- Histogramas ajudam, mas não são precisos sozinhos
 - Dependem do número de *bins*.
 - Não definem limites objetivos para identificação dos *outliers*.
 - Um *outlier* pode “sumir” em um *bin* largo.
- Eles são ótimos para uma visão geral, **sugerem a presença**, mas **não devem ser usados para a decisão final**.
- O que fazer então?

Como detectar *outliers*?

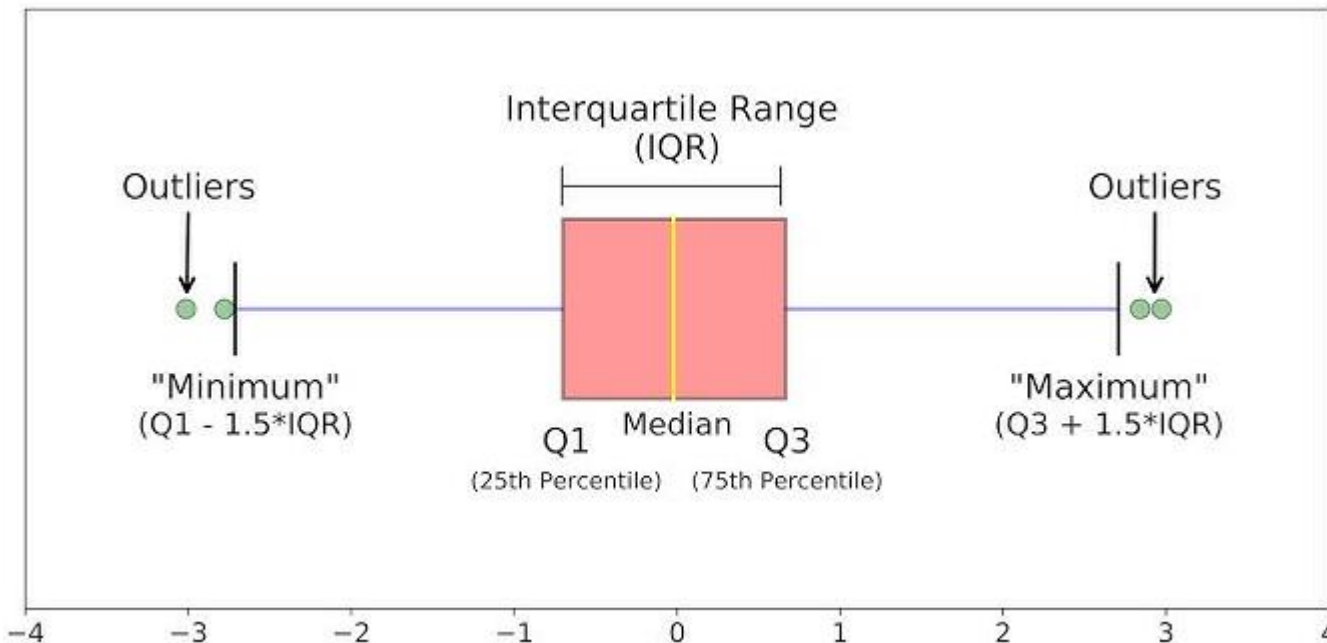
Boxplot



Como interpretar
o gráfico?

Como detectar *outliers*?

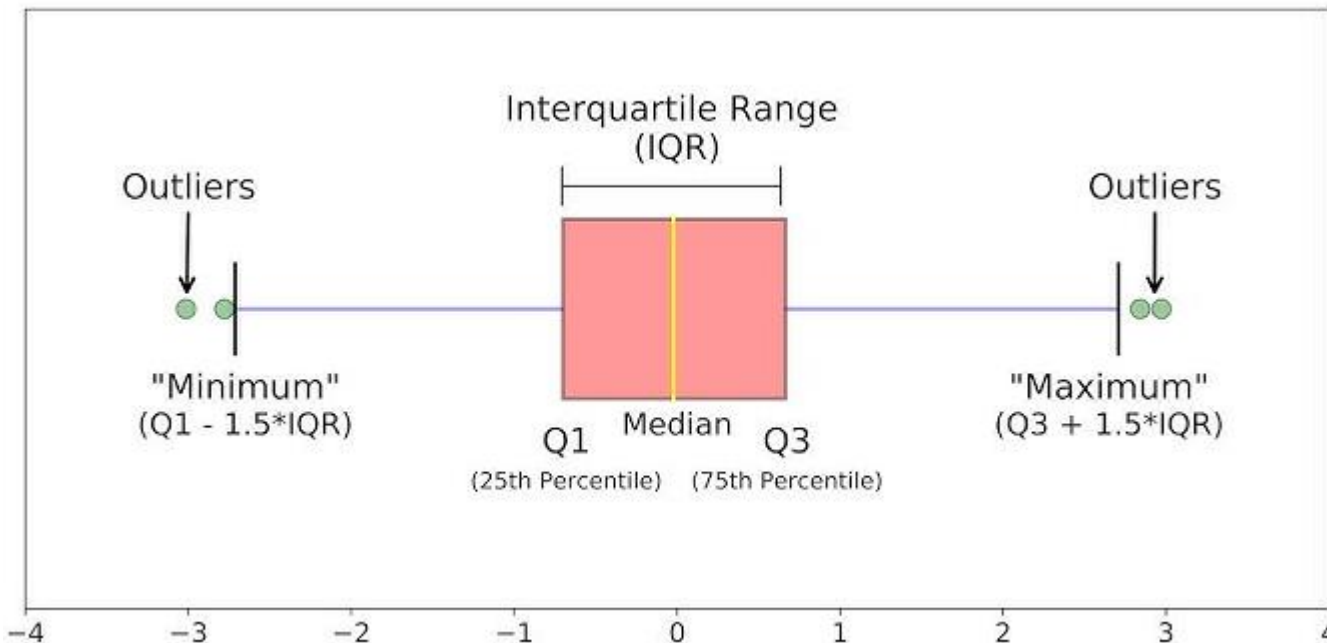
Boxplot



- A **caixa** central define o intervalo interquartil ($IQR = Q3 - Q1$).
- **Borda inferior da caixa** → primeiro quartil (Q1): 25% dos dados estão abaixo desse valor.
- **Linha dentro da caixa** → mediana (Q2): valor central dos dados. Metade dos valores está acima e metade abaixo.
- **Borda superior da caixa** → terceiro quartil (Q3): 75% dos dados estão abaixo desse valor.

Como detectar *outliers*?

Boxplot



- **Linhas que saem da caixa** → bigodes (*whiskers*): vão até o **menor e maior valores que não são outliers**.
 - Os limites são geralmente definidos por:
 - $Q1 - k \times IQR$
 - $Q3 + k \times IQR$
- **Pontos fora dos bigodes** → possíveis *outliers*.
- k controla a sensibilidade: $k = 1.5$ (**padrão**) para *outliers* moderados, e $k = 3.0$ para casos extremos.

Como detectar *outliers*?

Boxplot

- Usando a função **boxplot** da matplotlib

```
plt.boxplot(df[ 'Year' ])  
plt.xlabel( 'Year' )  
plt.ylabel( 'Valores' )  
plt.title( 'Boxplot do Year' )  
plt.show()
```

Vantagens e desvantagens: *boxplot* (IQR)

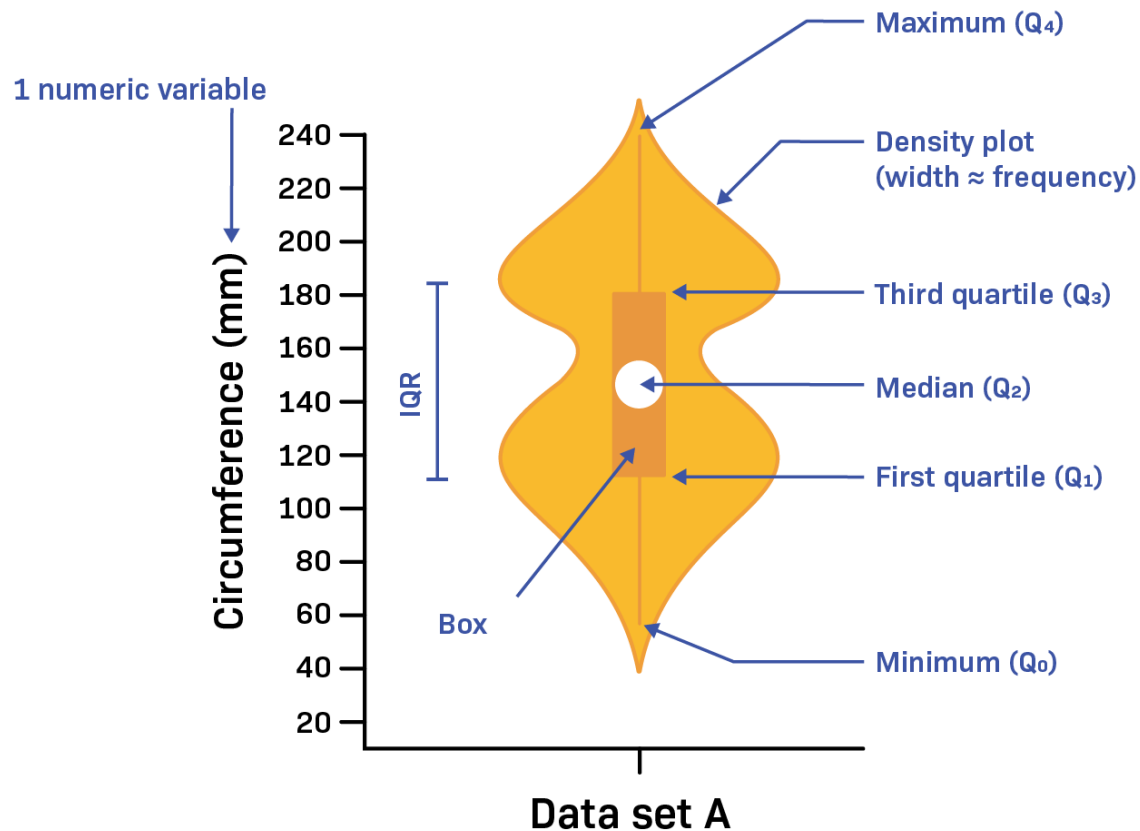
- Vantagens

- Robusto a valores extremos, pois usa quantis e não média
- Funciona bem mesmo com distribuições assimétricas
- Fácil de interpretar visualmente (*boxplots*)
- Não assume nenhuma distribuição específica dos dados

- Desvantagens

- Pode falhar em distribuições multimodais (vários picos)
- $k = 1.5$ é uma convenção prática e pode não ser ideal em todos os contextos.
 - Treine o modelo com e sem os *outliers* identificados por 1.5 e veja se a performance melhora.
 - Em conjuntos grandes, $k = 1.5$ pode definir muitos pontos como *outliers*, nesses casos, usar valor 3 ou ajustar empiricamente.
- Ineficaz para conjuntos de dados pequenos, pois eles não representam a distribuição com precisão

Outros métodos para detecção de *outliers*



- Além de IQR e Z-score, existem técnicas mais avançadas para detectar *outliers*, como:
 - Métodos usando algoritmos de ML: Isolation Forest, Local Outlier Factor, One-Class SVM, Autoencoders.
 - Métodos visuais: *Violin plot*
 - Combina *boxplot* com densidade das distribuições, destacando **assimetrias**.

Como remover *outliers*?

- **Aplicando a distância interquartil (IQR) em várias colunas numéricas**

```
num_cols = df.select_dtypes(include='number').columns
```

```
Q1 = df[num_cols].quantile(0.25)
```

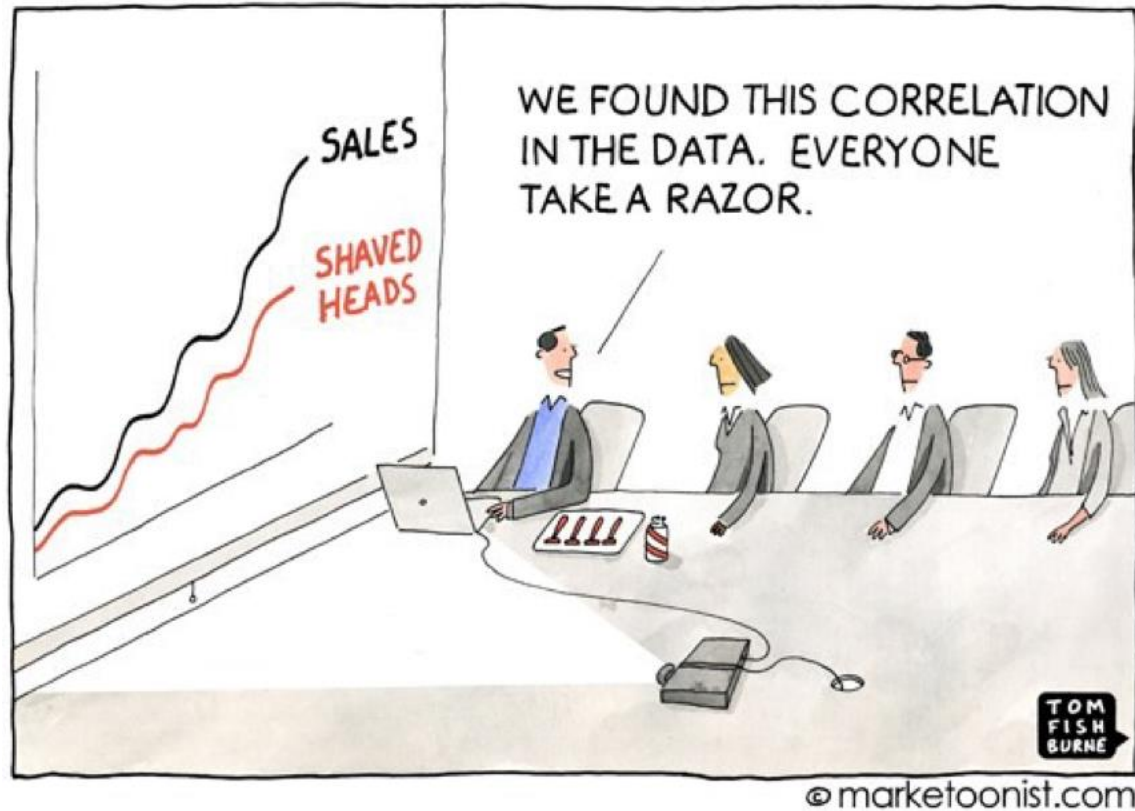
```
Q3 = df[num_cols].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df_sem_outliers_iqr = df[  
    ~((df[num_cols] < (Q1 - 1.5 * IQR)) |  
      (df[num_cols] > (Q3 + 1.5 * IQR))).any(axis=1)  
]
```

- **OBS.:** 'any(axis=1)' retorna True se **qualquer coluna daquela linha for outlier** (i.e., True). Assim, mantemos o dataset com o mesmo número de linhas.

Relação entre variáveis



Objetivo: descobrir **padrões e estruturas escondidas** nos dados.

Relações entre variáveis revelam:

- Tendências
- Dependências
- Agrupamentos

Relação entre variáveis

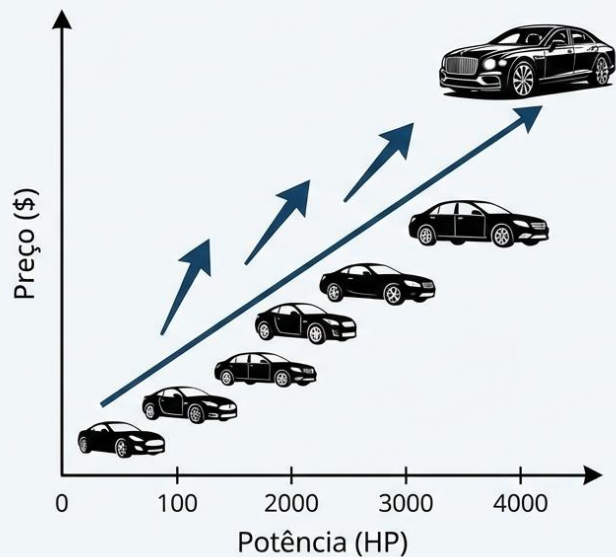


Importante!
Correlação não implica
causalidade

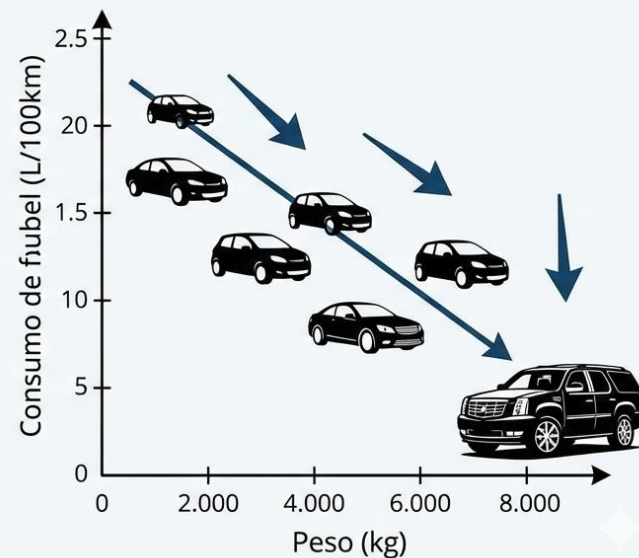
Comer frango não te faz ir
pra lua...

Relação entre variáveis

**Preço do carro ↑
conforme potência ↑**



**Consumo ↓
conforme peso ↑**



- Ao analisar relações, podemos responder:
 - O que realmente influencia o *target* (i.e., o rótulo)?
 - Essa variável faz sentido nesse contexto?
 - Esse comportamento é esperado ou estranho?

Exemplo: encontrar relações como

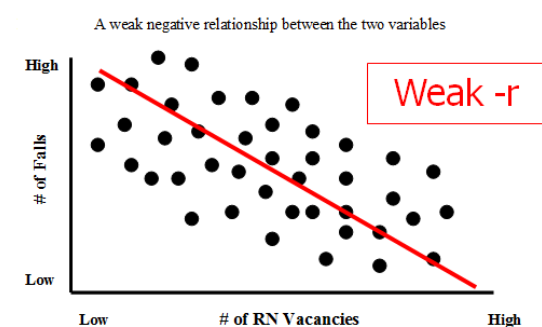
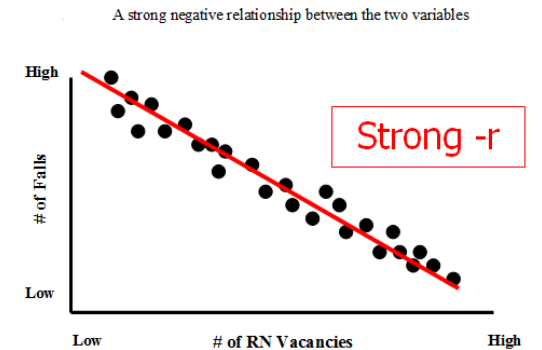
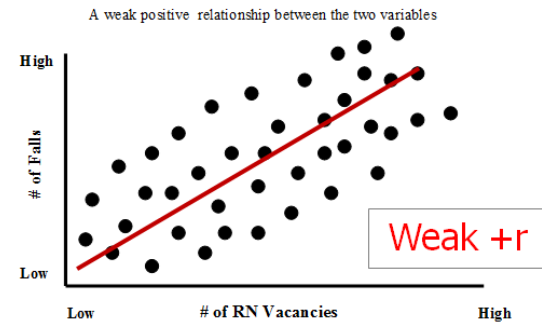
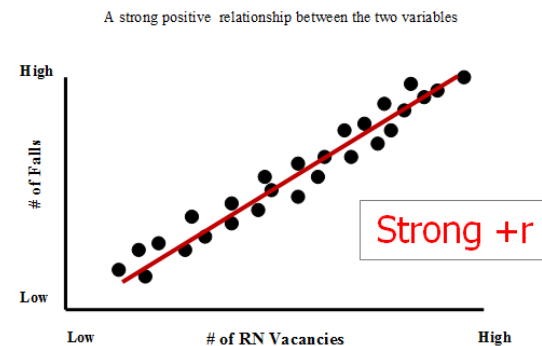
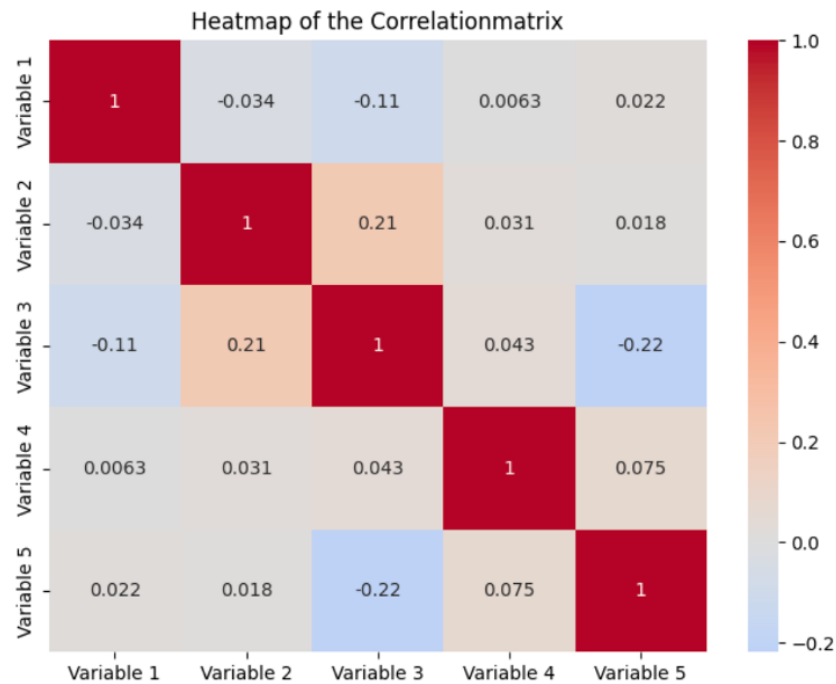
- Preço do carro ↑ conforme potência ↑
- Consumo ↓ conforme peso ↑

Relação entre variáveis

- Relações entre variáveis ajudam a:
 - selecionar atributos relevantes
 - remover atributos redundantes
 - detectar multicolinearidade
 - É quando **dois ou mais atributos estão altamente correlacionados** entre si, afetando negativamente a
 - ✓ interpretação dos coeficientes dos modelos,
 - ✓ generalização do modelo, levando a modelos instáveis,
 - ✓ predição dos modelos de ML.
 - escolher modelos de ML adequados

Relação entre variáveis

- Para analisar essas relações usamos:
 - Matriz de correlação (*heatmap*)
 - Diagrama de dispersão (*scatterplot*)



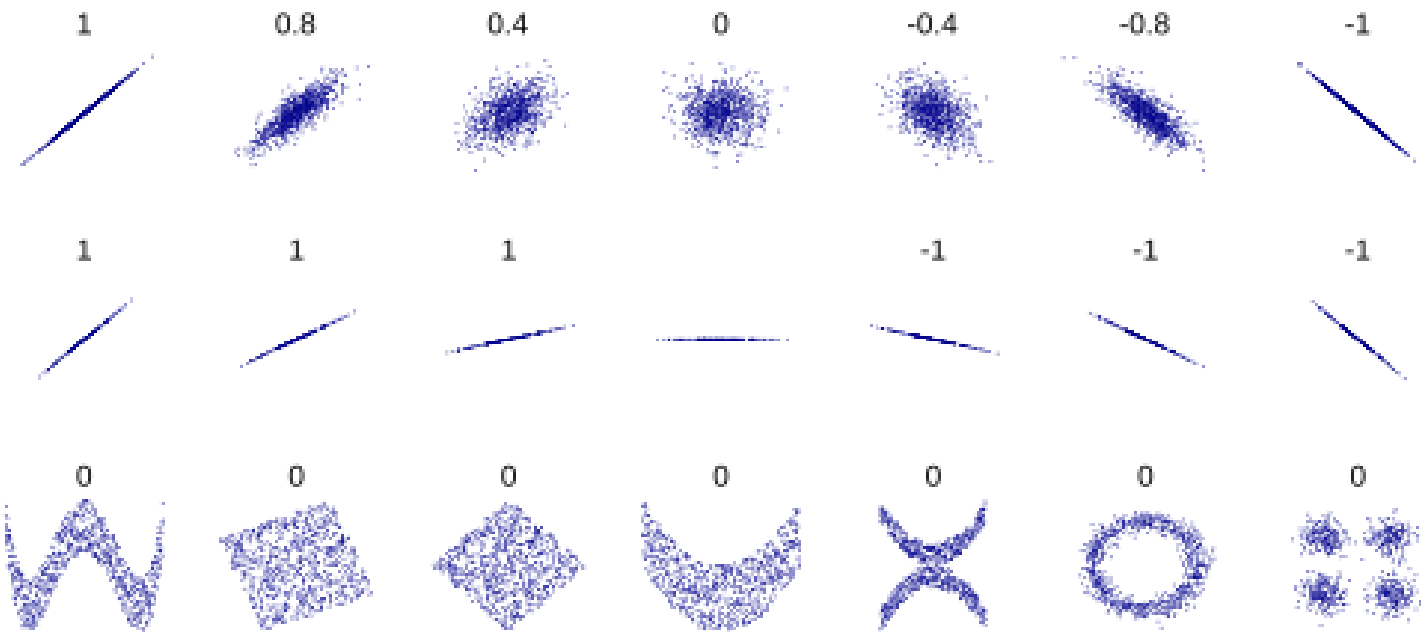
Matriz de correlação (*heatmap*)

- O *heatmap* mostra, de forma visual, o **grau de relação linear** entre **pares** de variáveis **numéricas**.
- Cada elemento da matriz é calculado usando-se o coeficiente de correlação de Pearson:

$$\rho = \frac{\sum_{i=1}^N (x_i - \mu_x)(y_i - \mu_y)}{\sqrt{\sum_{i=1}^N (x_i - \mu_x)^2} \sqrt{\sum_{i=1}^N (y_i - \mu_y)^2}} \approx \frac{\text{cov}(X, Y)}{\sqrt{\text{var}(X)\text{var}(Y)}}$$

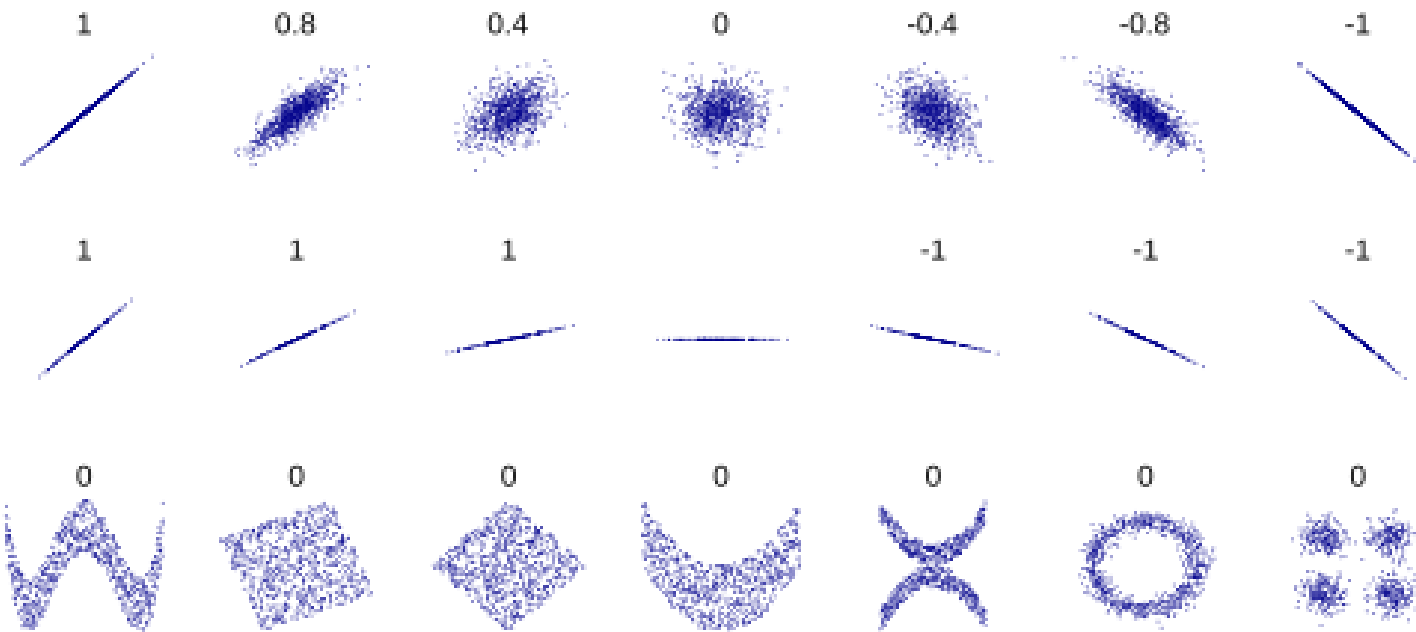
- Valores de ρ variam de -1 a $+1$

Correlação de Pearson



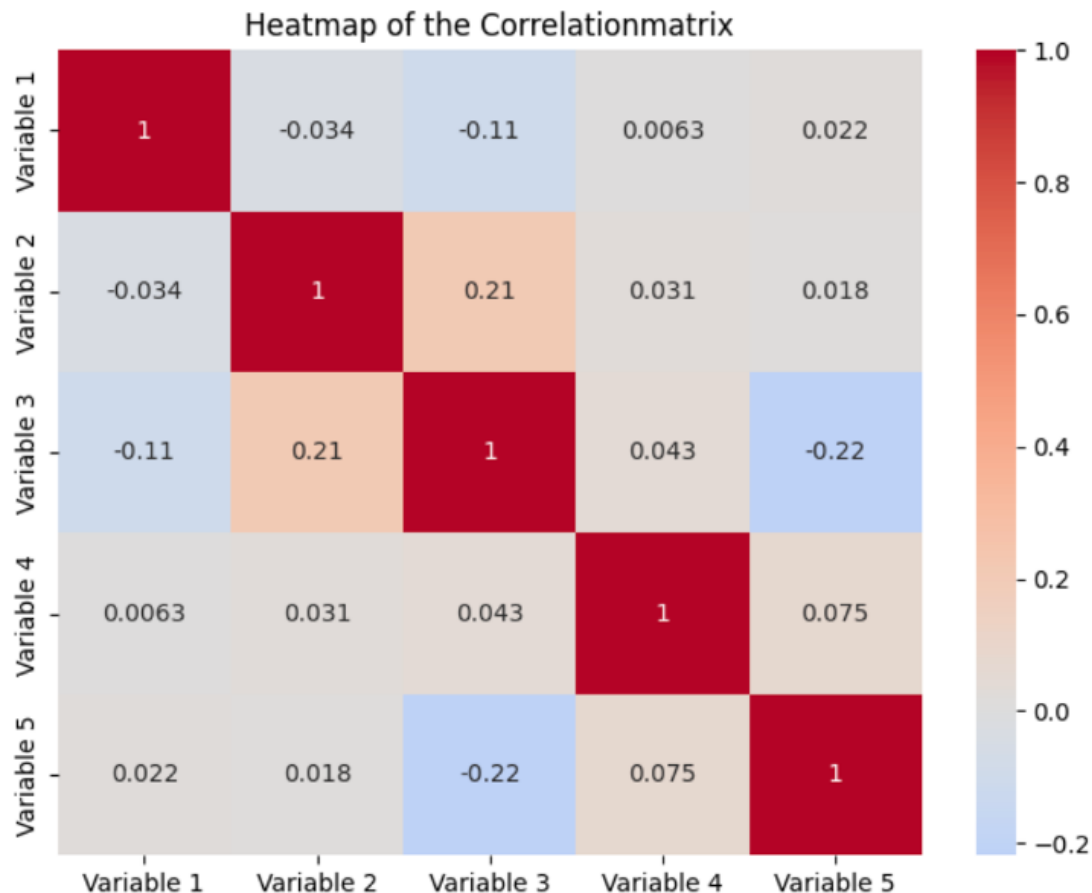
- $\rho > 0$: Quando uma variável aumenta, a outra também aumenta.
- $\rho < 0$: Quando uma variável aumenta, a outra diminui.
- $\rho = \pm 1$, pontos formam uma **linha reta perfeita**.
 - -1 relação inversa.
- $\rho = 0$: Os pontos estão espalhados como uma "nuvem" **sem direção clara**.
 - Não há relação linear entre as variáveis.

Correlação de Pearson



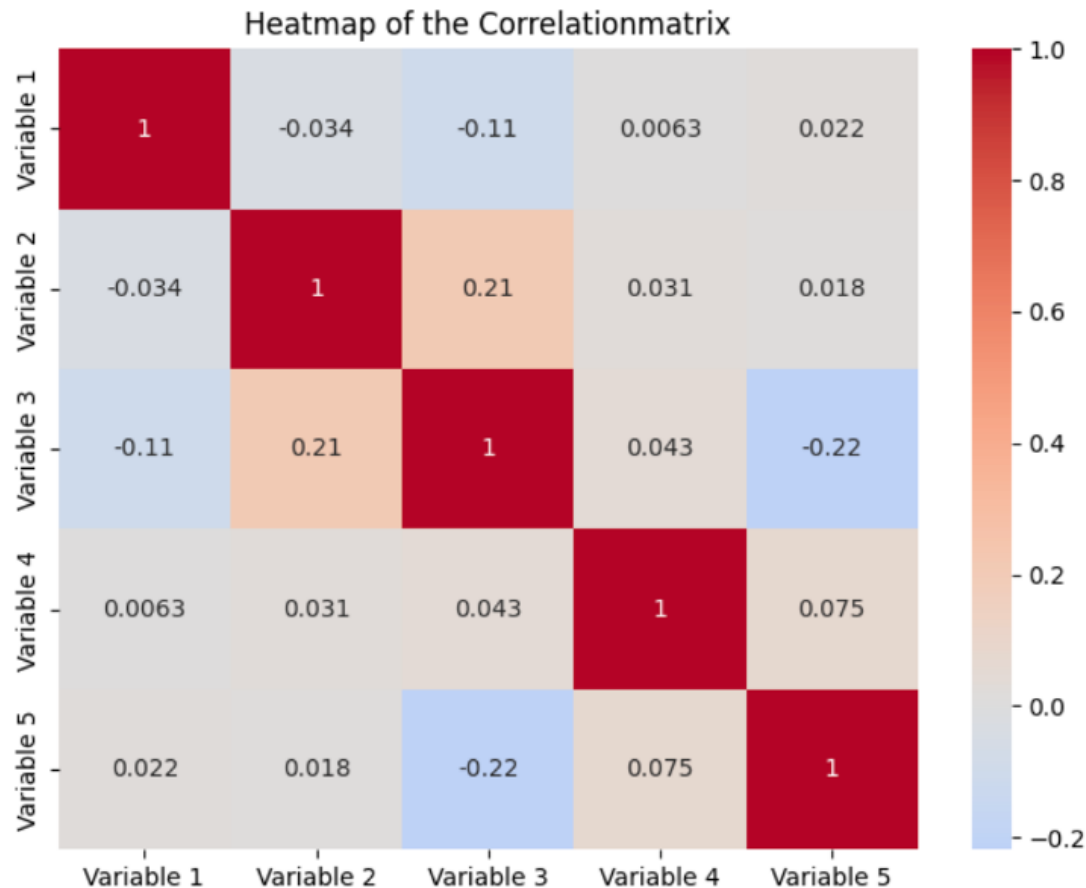
- ρ não depende da inclinação
 - Inclinações mais ou menos acentuadas resultam no mesmo ρ .
- ρ **também é igual a 0** se a correlação é **não-linear**
 - Mesmo y sendo uma função não linear de x (e.g., $y = \sin(x)$), ρ será 0.

Para que serve a matriz de correlação?



- Identificar variáveis fortemente relacionadas.
 - Relacionadas linearmente.
- Detectar multicolinearidade para **reduzir redundância**.
- Apoiar seleção de atributos.
 - Selecionar atributos que realmente influenciam no *target*.
- Levantar hipóteses sobre relações entre variáveis.
 - Relação pode ser modelada por uma reta?

Para que serve a matriz de correlação?



• Limitações

- ρ é sensível a *outliers* por usar a média em seu cálculo.
- **Não mostra a forma da relação** e, portanto, não indica se a **relação é não-linear**.
 - Devemos analisar visualmente (*scatter plot*) e/ou com outro coeficiente, como o de Spearman (Anexo IV).

Plotando a matriz de correlação

- **Matriz de correlação com o coeficiente de Pearson**

```
corr = df.corr(method='pearson', numeric_only=True)
plt.figure()
sns.heatmap(corr, cmap="BrBG", annot=True)
plt.show()
```

Diagrama de dispersão (*scatter plot*)

- **Mostra** a relação entre duas variáveis, **revelando padrões que números não mostram**.
- Permite visualizar:
 - tendências (positiva ou negativa),
 - relações não-lineares,
 - *clusters* (i.e., grupos) e
 - *Outliers*

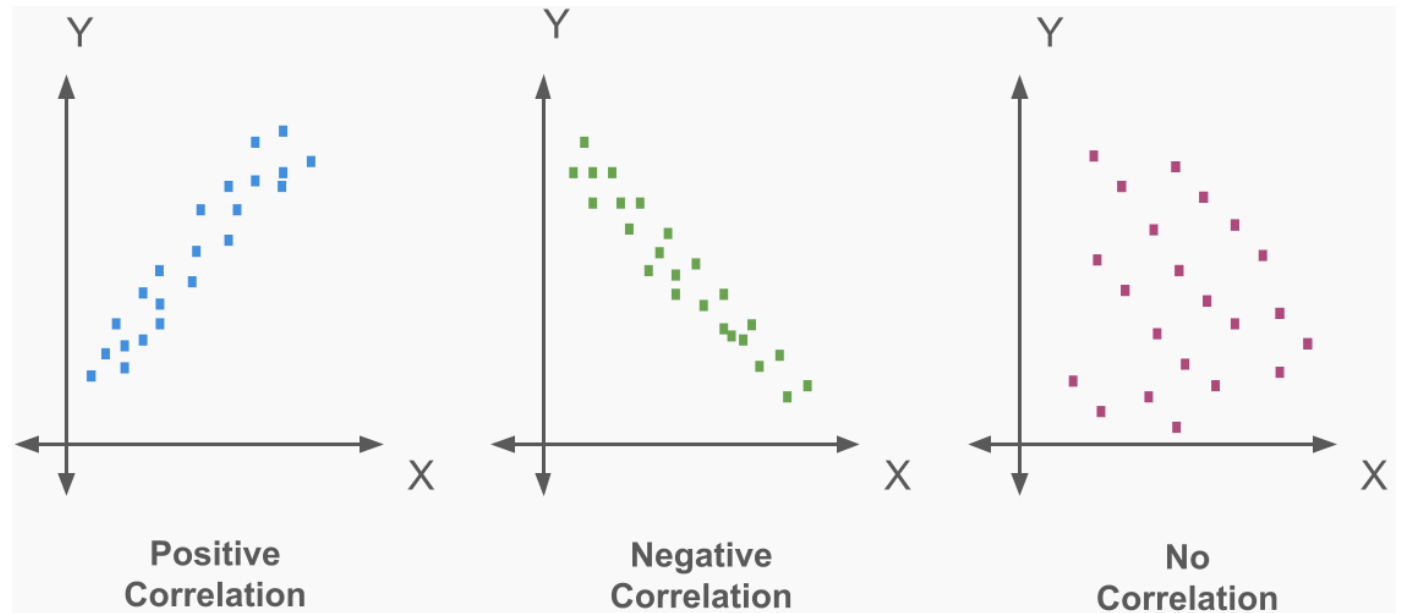


Diagrama de dispersão (*scatter plot*)

- Limitações:
 - Funciona bem para poucas variáveis
 - Quando o número de variáveis cresce, são necessários muitos *scatter plots*
 - A análise vira um **processo manual, cansativo e propenso ao erro**
 - Pode ficar poluído em *datasets* grandes:
 - os pontos se sobrepõem
 - regiões densas viram “manchas”
 - *outliers* podem ficar escondidos

Plotando o diagrama de dispersão

- **Plotando a dispersão de um par de variáveis**

```
plt.scatter(df['MPG-C'], df['Price'])  
plt.xlabel('MPG-C')  
plt.ylabel('Price')  
plt.show()
```

- **Plotando scatter plots para todos os pares de variáveis**

```
sns.pairplot(df)  
plt.show()
```

- **OBS.:** `pairplot` plota uma matriz de *scatter plots* para todos os pares de variáveis.

Exemplo

- [Exemplo: intro_eda.ipynb](#)



Perguntas?

Obrigado!

Anexo I:

Preenchimento inteligente de dados faltantes

Técnicas para o tratamento de valores ausentes

- **Preenchimento inteligente:**
- Usa estimativas dos valores faltantes obtidas através do padrão dos dados, não um valor fixo.
- O valor faltante é estimado a partir de outras colunas e linhas do *dataset*.
- Pode-se usar as classes da biblioteca SciKit-Learn
 - KNNImputer: usa a media das K amostras mais “próximas” para predizer os valores faltantes.
 - IterativeImputer: treina um modelo de regressão para cada atributo (i.e., coluna) para predizer os valores faltantes.
- **OBS.:** Funcionam apenas para dados numéricos.

Tratando valores faltantes

Exemplo de uso do KNNImputer

```
from sklearn.impute import KNNImputer

df_num = df.select_dtypes(include="number")

imputer = KNNImputer(n_neighbors=5) # número de vizinhos: 5
df_num_imputed = imputer.fit_transform(df_num)
```

OBS.1: O método 'fit_transform' treina o modelo e preenche os valores faltantes.

OBS.2: Em ML, calculamos o valor faltante **usando apenas os dados do conjunto de treinamento** (para evitar *data leakage*). Isso é válido para qualquer pré-processamento/transformação de dados.

Tratando valores faltantes

Exemplo de uso do `IterativeImputer`

```
# import necessário
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

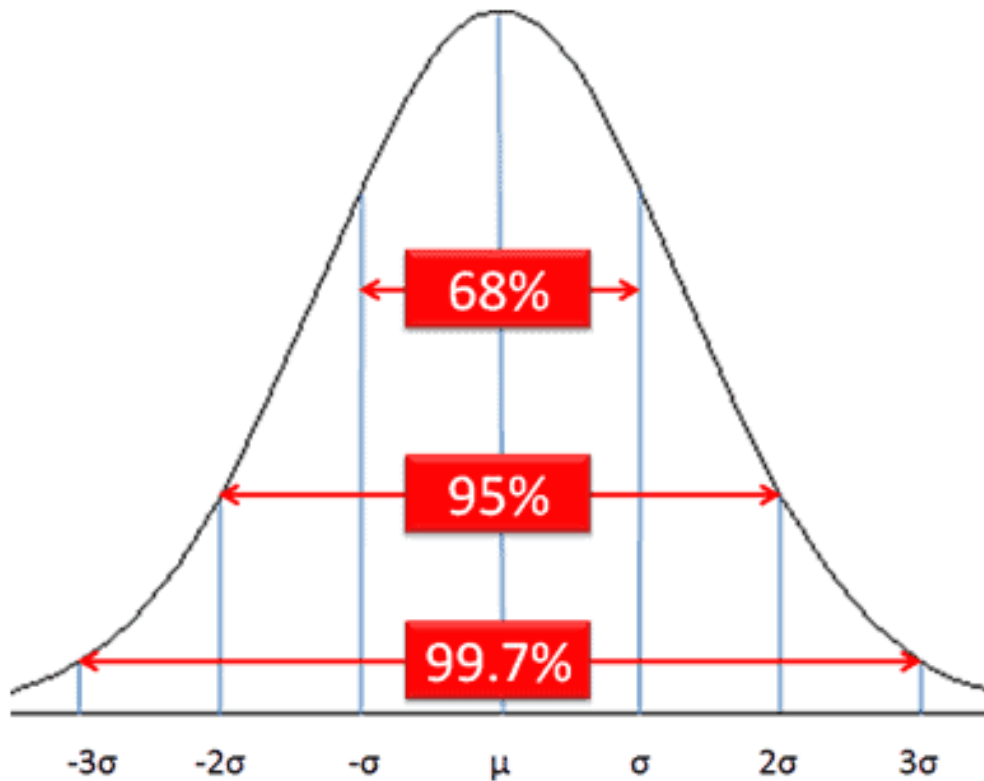
df_num = df.select_dtypes(include="number")

imputer = IterativeImputer()
df_num_imputed = imputer.fit_transform(df_num)
```

Anexo II:

Detectando *outliers* com Z-score

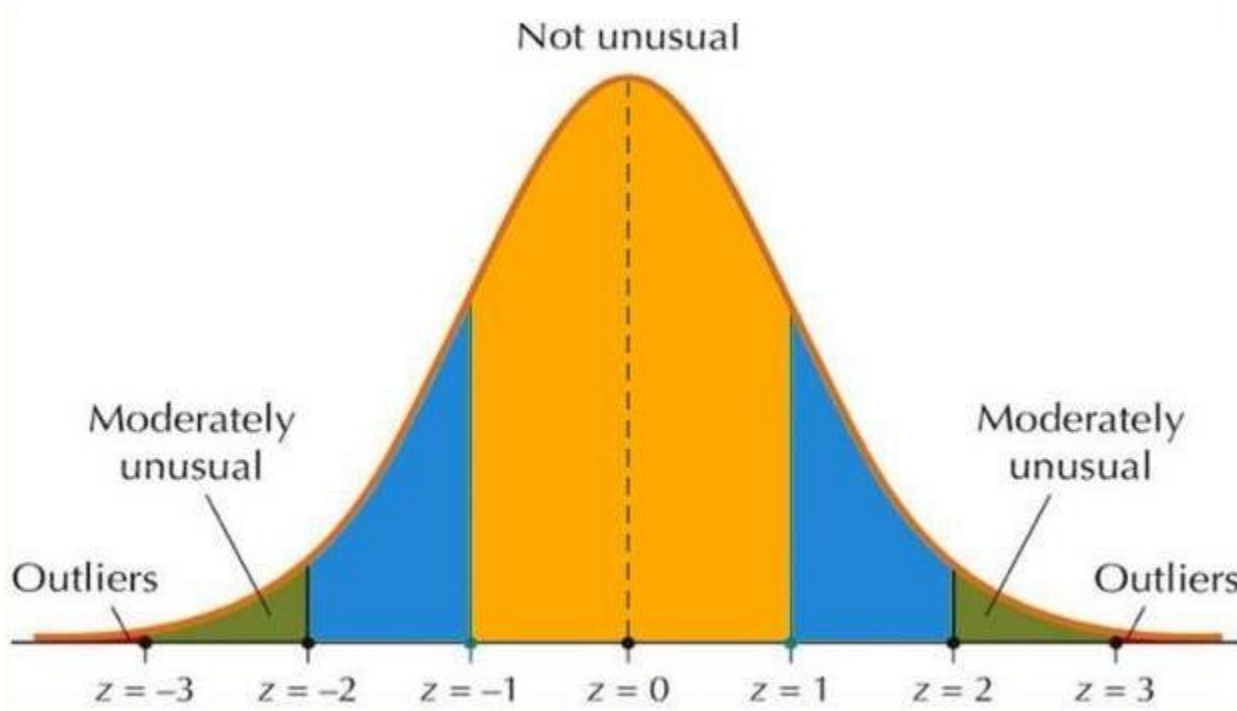
Como detectar *outliers*?



Z-score

- Mede quantos desvios-padrão um valor está distante da média da distribuição.
- Assume que os dados seguem uma distribuição semelhante à normal.
- 99.7% dos valores estão dentro do intervalo de $\pm 3\sigma$.
- Assim, assume-se que valores maiores do que $\pm 3\sigma$ são possíveis *outliers*, pois são valores raros.

Como detectar *outliers*?



Z-score

- Z-score de um valor x é calculado como:

$$z = \frac{x - \mu}{\sigma}$$

- z indica a distância até a média:
 - $z = 0 \rightarrow$ valor igual à média
 - $z = 1 \rightarrow$ 1 desvio-padrão acima da média
 - $z = 2 \rightarrow$ 2 desvios-padrão abaixo da média

Como detectar *outliers*?

Z-score

```
num_cols = df.select_dtypes(include='number').columns
z_scores = np.abs(stats.zscore(df[num_cols]))
outliers_por_linha = (z_scores > 3).any(axis=1)
df_outliers = df[outliers_por_linha]
```

Como remover *outliers*?

- **Aplicando Z-score em várias colunas numéricas**

```
num_cols = df.select_dtypes(include='number').columns
```

```
z_scores = np.abs(stats.zscore(df[num_cols], nan_policy='omit'))
```

```
df_sem_outliers_z = df[(z_scores < 3).all(axis=1)]
```

- **OBS.1:** Ignora valores NaN no cálculo da média e do desvio padrão.
- **OBS.2:** `'all(axis=1)'` retorna True somente se todas as colunas daquela linha tiverem $|z| < 3$, i.e., se qualquer variável da linha for *outlier*, a linha inteira é removida.

Vantagens e desvantagens: Z-score

- Vantagens

- Funciona bem quando os dados seguem uma distribuição aproximadamente normal
- Permite comparar atributos em escalas diferentes (dados são padronizados)
- Simples de calcular e interpretar

- Desvantagens

- Sensível a outliers, pois média e desvio-padrão são afetados por valores extremos
- Não funciona bem com distribuições assimétricas
- Pode gerar muitos falsos positivos em distribuições assimétricas
- Parâmetros são necessários: μ e σ da população devem ser conhecidos.
 - Caso contrário, devem ser usadas estimativas da amostra, o que pode ser um problema com *datasets* pequenos.

Anexo III:

Correlação de Spearman

Correlação de Spearman

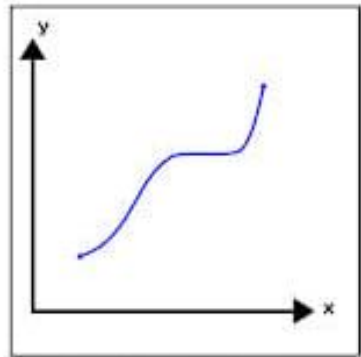


Figure 1 - A Monotonically Increasing function

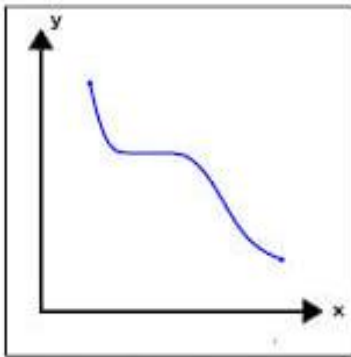


Figure 2 - A Monotonically decreasing function

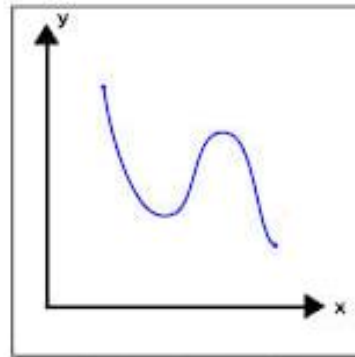


Figure 3 - A function that is not Monotonic

Se os dados crescem em formato de curva (exponencial, logarítmica), o Pearson dará um valor baixo, mas o Spearman dará 1.0.

- Mede a relação **monotônica** entre as variáveis.
 - Mede se quando uma variável sobe, a outra também sobe ou desce, independentemente se é em linha reta ou curva.
- Também avalia a relação de dados ordinais, i.e., dados que não são números exatos, mas têm uma ordem.
- Valor próximo de 0 se a relação for não-monotônica, i.e., se ela mudar de direção (e.g., $y = \sin(x)$).
- Menos sensível a *outliers*.

Plotando a matriz de correlação

- **Usando o coeficiente de Spearman**

```
corr = df.corr(method='spearman', numeric_only=True)
plt.figure()
sns.heatmap(corr, cmap="BrBG", annot=True)
plt.show()
```

Anexo IV:

Análise exploratória automática com pandas *profiling*

Pandas *profiling*

- O Pandas profiling (ou *ydata-profiling*) é uma biblioteca que automatiza grande parte da análise exploratória de dados (EDA) gerando um relatório completo com uma única linha de código.
- O relatório que inclui:
 - Resumo geral dos dados: número de linhas e colunas, tipos de dados.
 - Estatísticas univariadas: média, mediana, moda, desvios-padrão, histogramas.
 - Identificação de problemas: valores faltantes, duplicatas e potenciais anomalias.
 - Correlação e interações entre variáveis numéricas e categóricas.
 - Visualizações: de distribuições, barras, densidades e matrizes de correlação.
- O relatório pode ser salvo em HTML ou em um *notebook* Jupyter.

Como gerar o relatório?

```
from ydata_profiling import ProfileReport

profile = ProfileReport(df, title="Relatório de Dados")

# Salva em um arquivo HTML.
profile.to_file("relatorio.html")

# Abre o relatório no notebook Jupyter.
profile.to_notebook_iframe()
```